

# SQL Access Control Exercises - GameVerse Database

---

## Entry Level Database Security & Permissions

**Duration:** 45 minutes

**Database:** GameVerse (Game Store Database)

---

## Introduction to Access Control

Access control in databases determines **who can do what** with your data. Key concepts:

- **Roles/Users:** Database accounts with specific permissions
  - **Privileges:** Actions users can perform (SELECT, INSERT, UPDATE, DELETE, etc.)
  - **GRANT:** Command to give permissions
  - **REVOKE:** Command to remove permissions
  - **Principle of Least Privilege:** Give users only the permissions they need
- 

## Part 1: Understanding Roles and Users (10 minutes)

### Exercise 1.1: Create a Basic User

Create a new role called `game_reviewer` with login capability and password 'reviewer2026'.

► Hint

---

### Exercise 1.2: Create Multiple Roles

Create three roles for different user types:

- `customer_user` (password: 'customer123')
- `support_staff` (password: 'support123')
- `data_analyst` (password: 'analyst123')

► Hint

---

### Exercise 1.3: View Existing Roles

Write a query to see all roles in the database.

► Hint

---

## Part 2: Granting Read Permissions (10 minutes)

### Exercise 2.1: Grant Database Connection

Grant the `customer_user` role the ability to connect to the `gameverse` database.

► Hint

---

### Exercise 2.2: Grant Schema Access

Grant the `customer_user` permission to use the `public` schema.

► Hint

---

### Exercise 2.3: Grant SELECT on Specific Table

Grant the `customer_user` role SELECT permission on the `games` table only.

► Hint

---

### Exercise 2.4: Grant SELECT on Multiple Tables

Grant the `customer_user` role SELECT permission on `games`, `genres`, `publishers`, and `platforms` tables.

► Hint

---

### Exercise 2.5: Grant SELECT on All Tables

Grant the `data_analyst` role SELECT permission on all tables in the public schema.

► Hint

---

## Part 3: Granting Write Permissions (10 minutes)

### Exercise 3.1: Grant INSERT Permission

Grant the `game_reviewer` role the ability to INSERT new records into the `reviews` table.

► Hint

---

### Exercise 3.2: Grant UPDATE Permission

Grant the `game_reviewer` role the ability to UPDATE their own reviews in the `reviews` table.

► Hint

---

### Exercise 3.3: Grant Multiple Permissions

Grant the `support_staff` role the ability to SELECT, INSERT, and UPDATE on the `users` table.

► Hint

---

### Exercise 3.4: Grant DELETE Permission

Grant the `support_staff` role the ability to DELETE records from the `reviews` table (for moderation purposes).

► Hint

---

### Exercise 3.5: Grant Sequence Permissions

When inserting data, users need access to sequences (for auto-incrementing IDs). Grant the `game_reviewer` USAGE permission on all sequences in the public schema.

► Hint

---

## Part 4: Revoking Permissions (5 minutes)

### Exercise 4.1: Revoke SELECT Permission

Revoke the SELECT permission on the `users` table from the `customer_user` role.

► Hint

---

### Exercise 4.2: Revoke DELETE Permission

Revoke the DELETE permission on the `reviews` table from the `game_reviewer` role.

► Hint

---

### Exercise 4.3: Revoke All Permissions

Revoke all privileges on the `user_library` table from the `customer_user` role.

► Hint

---

## Part 5: Real-World Scenarios (10 minutes)

### Scenario 5.1: Read-Only Customer Access

You need to create a `readonly_customer` role that can:

- Connect to the database
- View games, genres, publishers, developers, and platforms
- NOT see sensitive user data or financial information

Write all necessary commands.

► Hint

---

### Scenario 5.2: Content Moderator

Create a `moderator` role that can:

- Connect to the database
- Read all data
- Update and delete reviews (for moderation)
- Update user account\_status (to suspend accounts)

Write all necessary commands.

► Hint

---

### Scenario 5.3: Game Library Manager

Create a **library\_manager** role that can:

- Connect to the database
- Read all game-related data
- Add, update, and remove games from user libraries (user\_library table)
- NOT modify game prices or user financial data

Write all necessary commands.

► Hint

---

### Scenario 5.4: Analytics Team Member

Create an **analytics\_user** role that can:

- Connect to the database
- Read all data from all tables
- Create temporary tables for analysis
- NOT modify any permanent data

Write all necessary commands.

► Hint

---

## Part 6: Checking Permissions

### Exercise 6.1: View Table Permissions

Write a query to view all permissions granted on the **reviews** table.

► Hint

---

### Exercise 6.2: View User's Permissions

Write a query to see all privileges granted to the **game\_reviewer** role.

► Hint

---

## Challenge Questions

## Challenge 1: Security Audit

Explain why it's dangerous to grant ALL PRIVILEGES to regular users. What could go wrong?

---

## Challenge 2: Column-Level Security

Research how to grant UPDATE permission on only specific columns (e.g., allow updating email but not is\_premium status). Write an example.

---

## Challenge 3: Row-Level Security

If you wanted users to only see/modify their own reviews, what PostgreSQL feature would you use? (Research: Row-Level Security policies)

---

## Best Practices

1. **Least Privilege:** Only grant the minimum permissions needed
  2. **Use Roles:** Group permissions into roles, then assign roles to users
  3. **Audit Regularly:** Check who has access to what
  4. **Separate Duties:** Don't give one person all powers
  5. **Document:** Keep track of who has what permissions and why
  6. **Remove Unused Access:** Revoke permissions when no longer needed
- 

## Security Reminders

- Never share database passwords
  - Use strong passwords in production
  - Regular users should NOT have DROP, TRUNCATE, or ALTER permissions
  - Protect sensitive data (emails, financial info) with strict access controls
  - Monitor database activity for suspicious behavior
- 

**Good luck!**